TECHNION
Israel Institute of Technology

DEPARTMENT OF CHEMICAL ENGINEERING
TECHNION – ISRAEL INSTITUTE OF TECHNOLOGY

WEEK 01 · LECTURE · 00560413

AI Fluency and Agentic Methods for Engineers

Lecture 1

Course intro

Assoc. Prof. Alon Grinberg Dana

Department of Chemical Engineering · Technion

SLIDO · QUESTIONS, ALL LECTURE

Ask any time.

Scan the code, or open app.sli.do and join this event.

I take questions at the end of each block.
Vote up the ones you want answered.
Remind me if I forget!



Outline

01	**Syllabus**	the course, the weeks, the milestones	←
02	Concept	the 10× claim, three approaches, your environment	
03	git	organizes and protects the code	
04	Live demo	a real example	

Syllabus

00560413 · 2.5 credit points · Fall 2026-7

Lectures	Monday 08:30–10:30 · Ulman 803
Exercises	Monday 10:30–12:30 · Ulman 703 and 801, in parallel
Lecturer	Assoc. Prof. Alon Grinberg Dana · `alon@technion.ac.il`
TAs	Ori Goldfryd · Ramzy Ayan
Reception	By appointment, over email
Prerequisites	`02340128` Python · `00540330` Simulation Lab, or an approved equivalent. Basic software engineering

Every week: lecture 2 h, exercise 2 h, homework 4 h. Bring your own laptop.

Syllabus

What the course is for

Every graduate ships at least one **working agentic system** that solves a real problem in their own field.

The science

What an LLM is, and how it differs from a deterministic numerical method

The stack

Version control, code review, type checking, tests, CI

By hand first

Do it yourself before you delegate it, so you can verify the agent

The system

An agent that drives a simulator toward a better design, under constraints and human checkpoints

Offered by Chemical Engineering, open across the Technion. Case studies adapt to your domain.

Syllabus

Thirteen weeks, six milestones

W1	Intro: approaches, environment, git	**W8**	Connecting tools **M4**
W2	LLM science **M1**	**W9**	Human in the loop
W3	The agent: harness + LLM	**W10**	Discernment and verification **M5**
W4	Containment **M2**	**W11**	Open
W5	Context engineering	**W12**	Open **M6**
W6	Elicitation: grilled into a spec **M3**	**W13**	Showcase and live defense
W7	Tests as the control surface		

Your project moves one increment a week. A milestone is due every two weeks.



Syllabus

The six milestones

M1	W2	Problem statement · GitHub user registered with course staff · first merged PR
M2	W4	First agent-authored commit
M3	W6	Agent environment file · domain glossary · one written skill · written specification *Last chance to revise the project*
M4	W8	One real engineering tool connected and in use · specification-derived tests green in CI
M5	W10	Declared intervention points · structured logging One induced failure diagnosed from the log alone · first evaluation run
M6	W12	Evaluation suite · peer-review on GitHub · canonical run Demo · tagged release · defense pack

Syllabus

Graded on the project. No written final exam

10%	40%	25%	20%	5%
Setup, W2	Working product	Demo and live defense, W13	Written report	Peer-review quality

Using AI is **required**, not merely permitted.
Graded: whether you **directed it, verified it, and can account for it**.

Every submission is rerun from a clean clone. The W13 defense is a random selection.
Work you cannot account for is an integrity failure.



Syllabus

Where things live

Course website	technionaifluency.github.io/www Lectures, exercises, schedule, pre-work, the project
Moodle	moodle26.technion.ac.il Forum and messages
GitHub	github.com/TechnionAIFluency Course code and templates. You work in your own fork

Course website

WORTH YOUR TIME

Anthropic	Building effective agents · anthropic.com/engineering

Syllabus

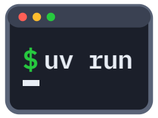
Pre-work, before the first exercise

Not taught in the course, not graded on its own. Week 1 assumes it.



Python you can read

Week 1 reads agent-written Python and reviews a diff. Following it is enough.



A real Unix shell

macOS and Linux: already there. Windows: WSL2, about twenty minutes.



uv

Install uv itself. The environment is built in the Week 1 exercise.



GitHub, with 2FA

All work is a pull request. Registering the account is part of M1, Week 2.

Every step, with links: `technionaifluency.github.io/www/prework`

Gracie

A full-duplex voice agent

There is **an agent in the room.**

Listens

Continuously, for the whole lesson

Intervenes

When asked, or with permission

Keeps a transcript

Text, of each lesson, for internal use

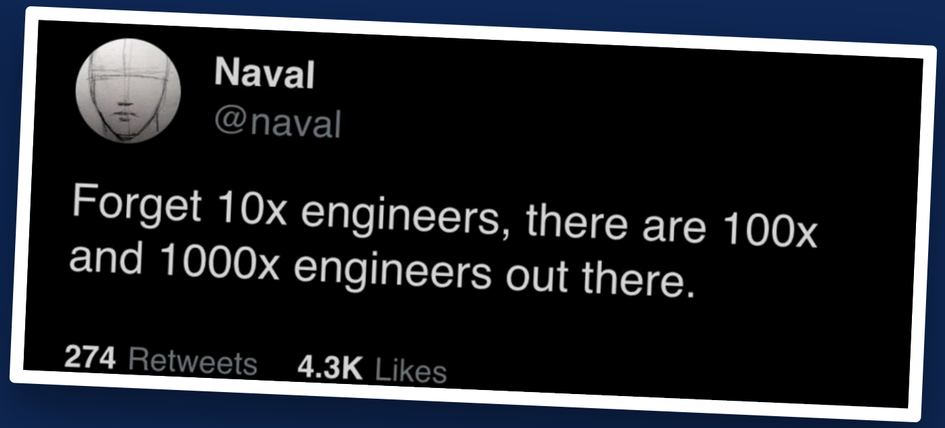
What is said is sent as text to Anthropic (Claude). No audio recording is saved. A text transcript of each lesson is saved. Gracie’s microphone hears the whole room, including students.

Outline

01	Syllabus	the course, the weeks, the milestones
02	**Concept**	the 10× claim, three approaches, your environment ←
03	git	organizes and protects the code
04	Live demo	a real example


Who is the Future
10x Engineer ?
in the Age of AI?


CAUTION:
10X
ENGINEER


Naval
@naval
Forget 10x engineers, there are 100x and 1000x engineers out there.
274 Retweets
4.3K Likes


10X
ENGINEER


10X ENGINEER
IN DA HOUSE


!10x


10x
ENGINEER

“AI makes engineers ten times more productive.”

A model, plus a tested ground, plus a loop, can search a space you could not search by hand.

The model

judgement about where to look

The ground truth

The loop

makes the two compound

Accelerate your work at the tactical layer, so you can spend more time in the **strategic layer**.

Three approaches

A Ask the model

B Write a script

C A model reasoning over a tested ground

Three ways to point a model
at a hard engineering problem.

Three approaches

Approach A: ask the model

A Ask the model

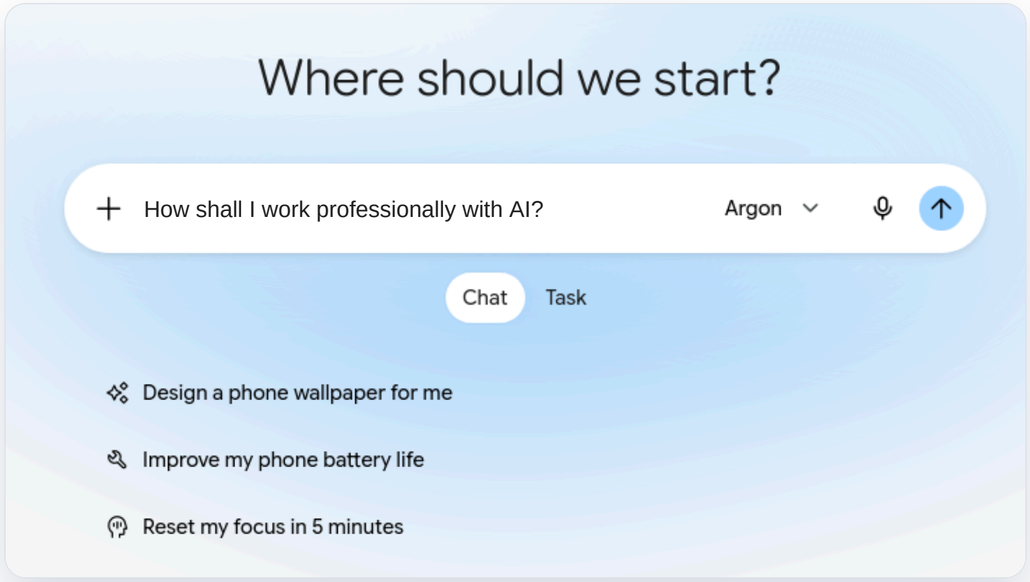
B Write a script

C A model reasoning over a tested ground

You ask. It answers.
You trust it, or you don't.

The model **reasons and computes**.
Fast. **Unverifiable. Not reproducible.**

Fine for low-stakes work.
Do not use this approach for professional work.



Three approaches

Approach B: write a script

A Ask the model

B Write a script

C A model reasoning over a tested ground

The model writes a deterministic, tested program. Present at **build time**. Gone at **run time**.

Honest engineering

Reproducible. Reviewable. Verifiable.

The limit

It only ever does the thing you thought of in advance.

Three approaches

Approach C: a model reasoning over a ground

A Ask the model	B Write a script	C A model reasoning over a tested ground

Build the tested ground first.
Then put a model **above it**, at run time.

It picks a strategy, calls the ground,
reads the numbers, adjusts, and loops.

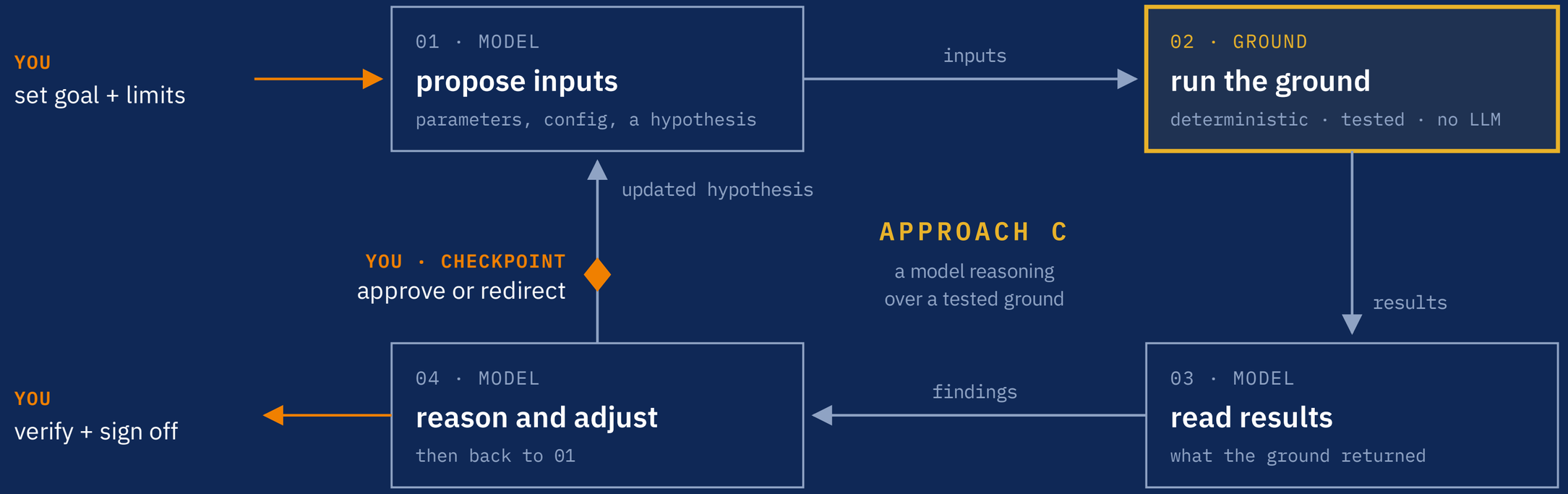
Every number it reports
came from code you tested.

Its unreliability does not vanish. It **moves**.

That is the course's subject.

The Approach C loop

Only step 02 has no LLM.



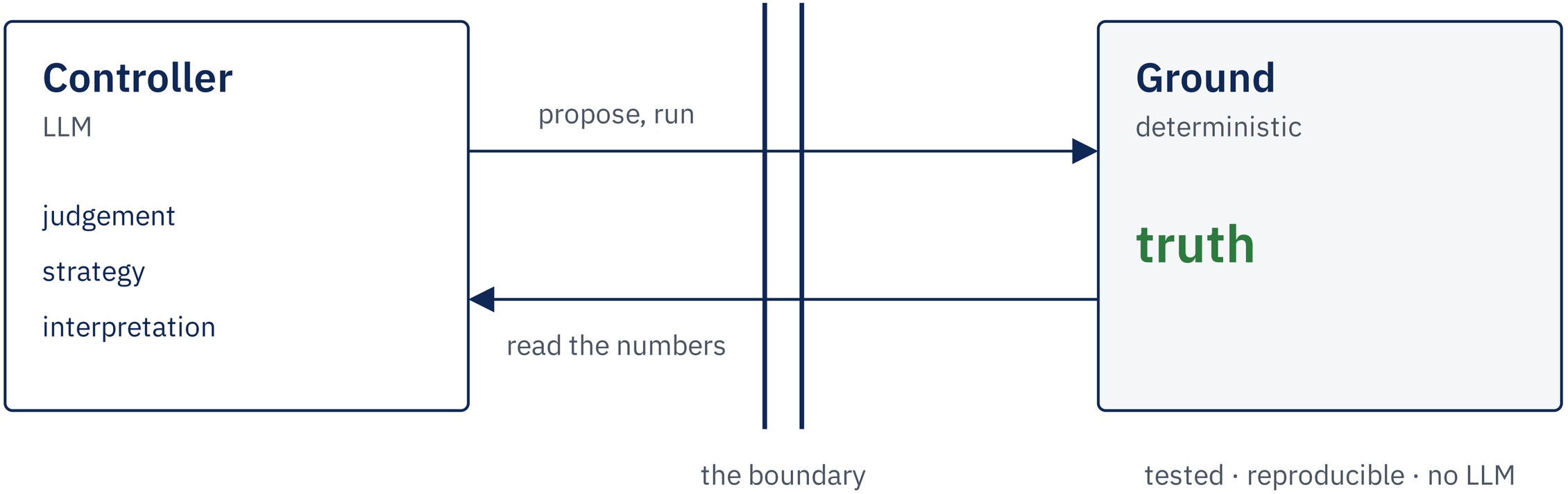
One task, three approaches

EXAMPLE TASK

Find the three input settings of a simulator that maximise its output without breaking any of its limits.

	WHAT YOU DO	WHAT YOU GET
A	**Ask for the best settings**	Three plausible numbers, no way to test this is indeed optimal
B	**Write an optimiser (adaptive grid, Bayesian, ...)**	The best point **the strategy you chose** can find, decided before it runs
C	**Let a model choose each next setting, and run it**	A reasoned search that can change strategy: every number computed, the path recorded

Approach C


Controller
LLM
judgement
strategy
interpretation
propose, run
read the numbers
Ground
deterministic
truth
the boundary
tested · reproducible · no LLM

The failure test

Too much on the model

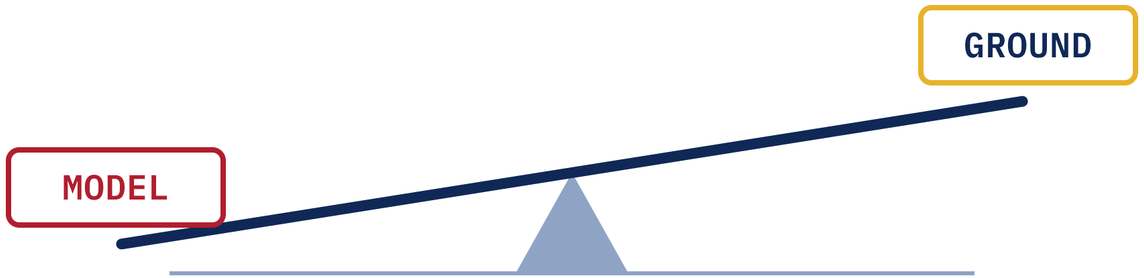
Hallucination, and nothing reproducible.

Too much on the ground

An ordinary program, and no agent.

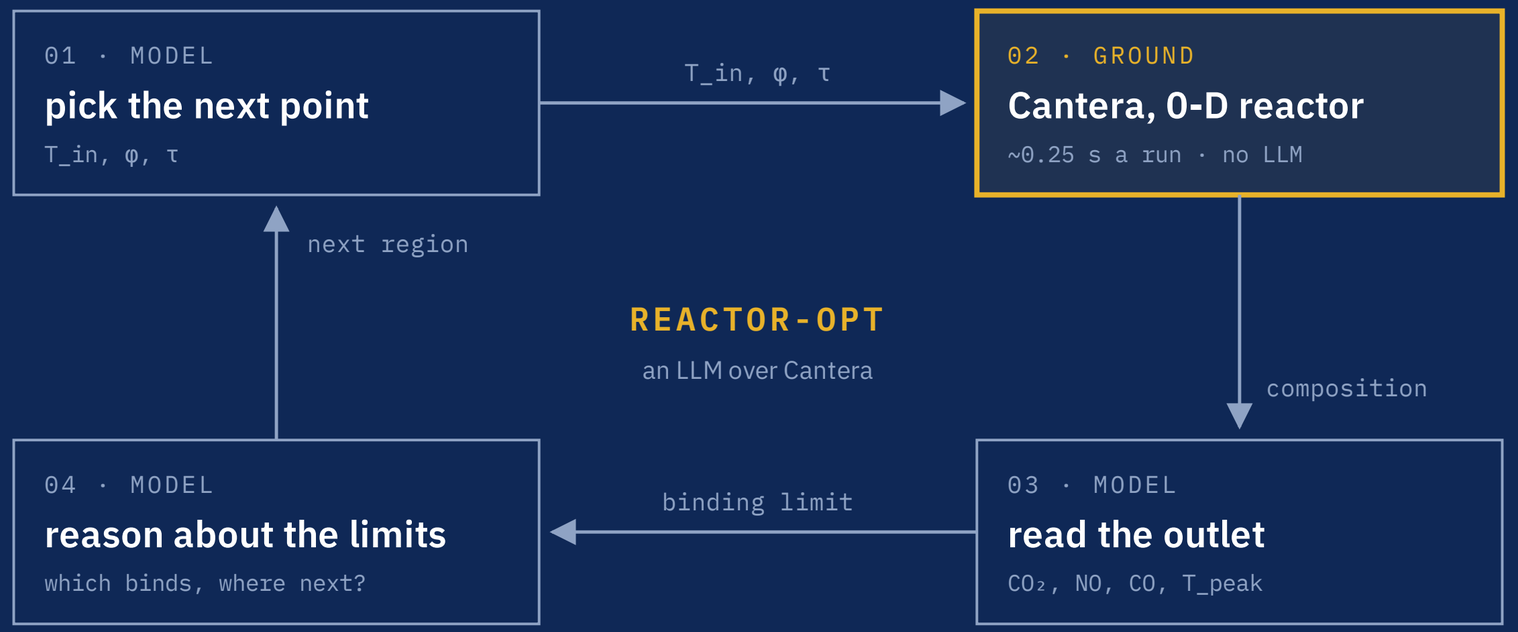
If you can delete the model, drop in a `for` loop, and it performs just as well, you built Approach B with a chat interface on top.

A real engineering artefact. Also a failing project.



The reference project: reactor-opt EXAMPLE 1 OF 3

This course · a methane/air reactor, three knobs



Goal

Maximise outlet CO_2 subject to $NO \le 5\times10^{-3}$, $CO \le 2\times10^{-2}$, $T_{peak} \le 2600$ K.

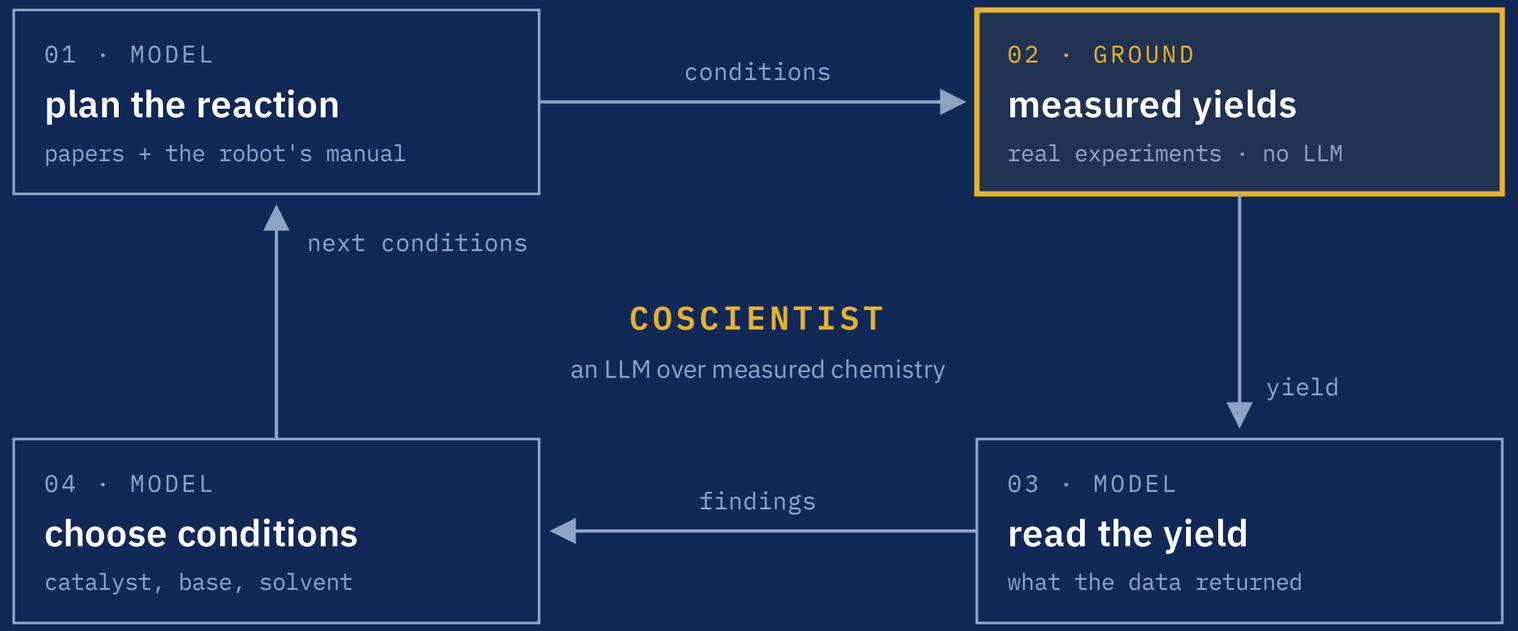
Your project lives here.

Delete the model

A grid can find it in 1,980 runs, if you aim it right.

A success: Coscientist EXAMPLE 2 OF 3

Carnegie Mellon, 2023 · a chemistry lab run by GPT-4



Result

Planned and ran Pd cross-couplings on a robot. Raised yield within 20 steps over measured data.

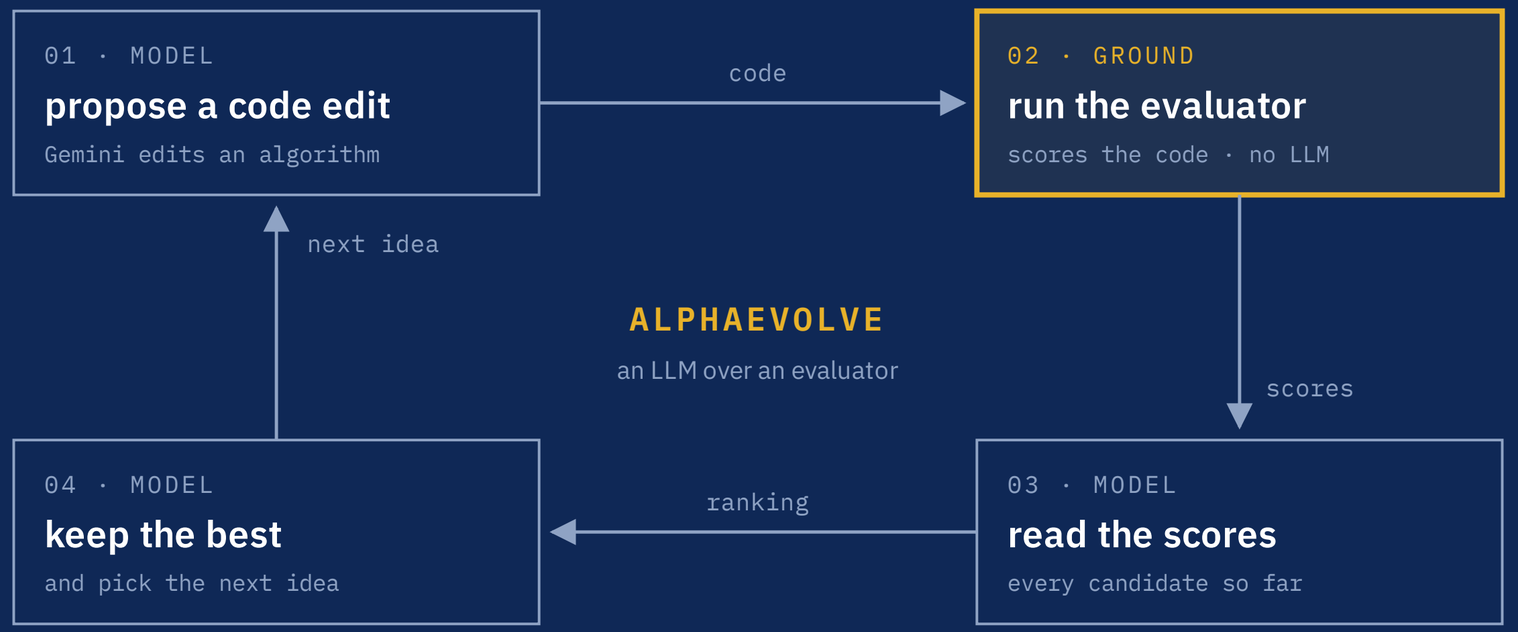
Delete the model

Bayesian optimisation, no LLM, on the same task: GPT-4 scored higher, and the authors say that is not conclusive. Step 01 is what the model clearly added.

Boiko, MacKnight, Kline, Gomes, Nature 624, 570 (2023)

A success: AlphaEvolve EXAMPLE 3 OF 3

Google DeepMind, 2025 · improving algorithms



Result

4×4 complex matrices multiplied with **48** multiplications.
The record since Strassen, 1969: 49.

Delete the model

Random edits in a `for` loop:
56 years of people did not find it either.

This is **Google-scale**. Your project runs the same loop at a much more reasonable scale.

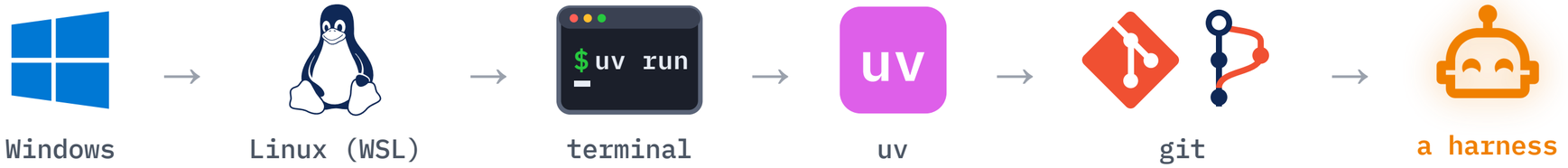
Novikov et al., AlphaEvolve, arXiv 2506.13131 (2025)

The environment, as one stack

A harness
`git`
A uv environment
The Linux terminal
`WSL` on Windows

Everything else in the course runs **on top** of this stack.

The first exercise gets it onto your machine and checks that it **works**.



The harnesses



Claude Code
Anthropic



Antigravity
Google



Codex CLI
OpenAI



OpenCode
open source

The model **thinks**. The harness **acts**.

A harness wraps a model with tools and runs the loop around it.

In Approach C the loop lives in a harness, and the ground sits behind a boundary it can call.

Outline

01	Syllabus	the course, the weeks, the milestones
02	Concept	the 10× claim, three approaches, your environment
03	**git**	organizes and protects the code ←
04	Live demo	a real example

git: the photo album

a commit is a photograph of your project

your folder

README.md notes.txt

the album = history




```
$ git switch --detach b3ced92
HEAD is now at b3ced92 Add README
```

Back to the first photo: sim.py did not exist yet.

You can put your folder back exactly as in any photo.

frame = staging · **photo** = commit · **album** = history

git: staging, the golf tee

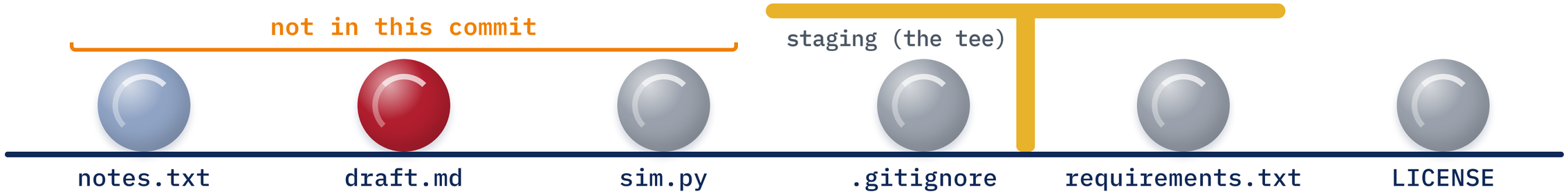
staging is choosing what goes into the next commit

```
$ git status --short
?? .gitignore
?? LICENSE
?? requirements.txt
$ git add .gitignore requirements.txt LICENSE
$ git status --short
A  .gitignore
A  LICENSE
A  requirements.txt
$ git commit -m "Project scaffolding"
[main 4c821e6] Project scaffolding
```

history

85ccc1c
Add the simulator

4c821e6
Project scaffolding



One swing, one commit.

Unlike golf, the files stay on the floor: a commit is a copy.

git, for real: commits on main

your folder · the terminal · the history

your folder **main**

reactor-study/
- README.md
- sim.py

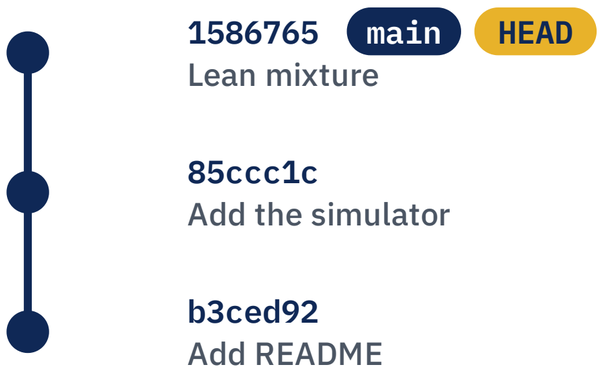
sim.py

```
T_IN = 1200
PHI = 0.9

def simulate(t_in, phi):
    ...
```

```
$ git init
$ git add README.md
$ git commit -m "Add README"
[main (root-commit) b3ced92] Add README
1 file changed, 1 insertion(+)
create mode 100644 README.md
$ git add sim.py
$ git commit -m "Add the simulator"
[main 85ccc1c] Add the simulator
1 file changed, 5 insertions(+)
create mode 100644 sim.py
$ git add sim.py
$ git commit -m "Lean mixture"
[main 1586765] Lean mixture
1 file changed, 1 insertion(+), 1
deletion(-)
```

history (git log)



git add chooses. git commit records. HEAD follows the newest commit.

git, for real: a branch

same sequence, one branch further

your folder — feature

```
reactor-study/
  README.md
  sim.py
  plot.py
```

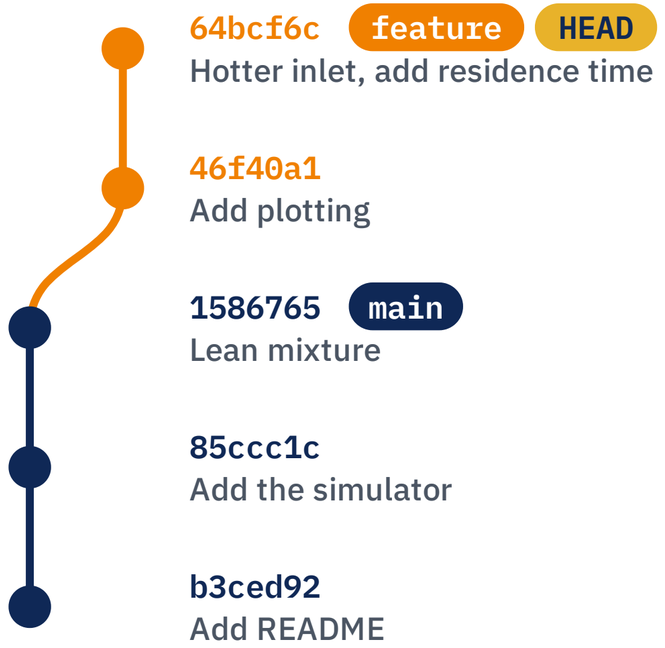
sim.py

```
T_IN = 1300
PHI = 0.9
TAU = 0.05

def simulate(t_in, phi):
    ...
```

```
[main 1586765] Lean mixture
$ git switch -c feature
Switched to a new branch 'feature'
$ git add plot.py
$ git commit -m "Add plotting"
[feature 46f40a1] Add plotting
1 file changed, 4 insertions(+)
create mode 100644 plot.py
$ git add sim.py
$ git commit -m "Hotter inlet, add
residence time"
[feature 64bcf6c] Hotter inlet, add
residence time
1 file changed, 2 insertions(+), 1
deletion(-)
```

history (git log)



A branch is a name. main stayed at 1586765 while feature moved on.

git: the switch

the folder changes under your feet

your folder — feature

reactor-study/
- README.md
- sim.py
- plot.py

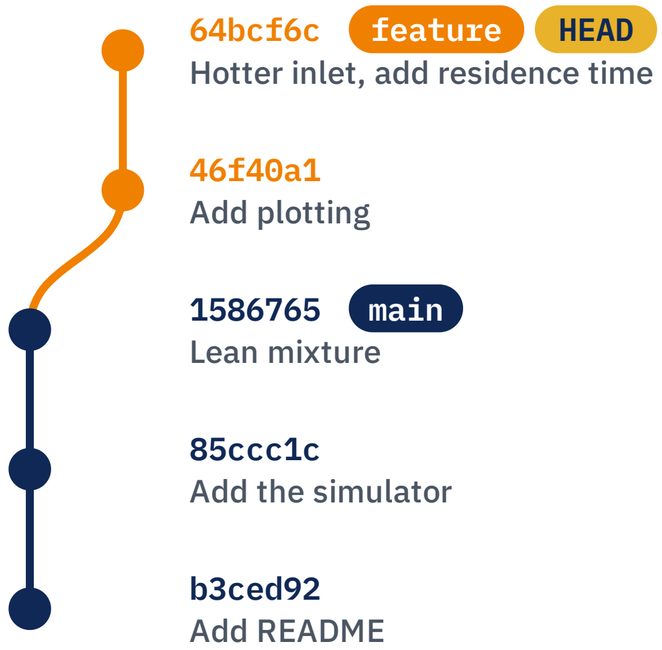
sim.py

```
T_IN = 1300
PHI = 0.9
TAU = 0.05

def simulate(t_in, phi):
    ...
```

```
$ git switch main
Switched to branch 'main'
$ ls
README.md
sim.py
$ git switch feature
Switched to branch 'feature'
$ ls
README.md
plot.py
sim.py
```

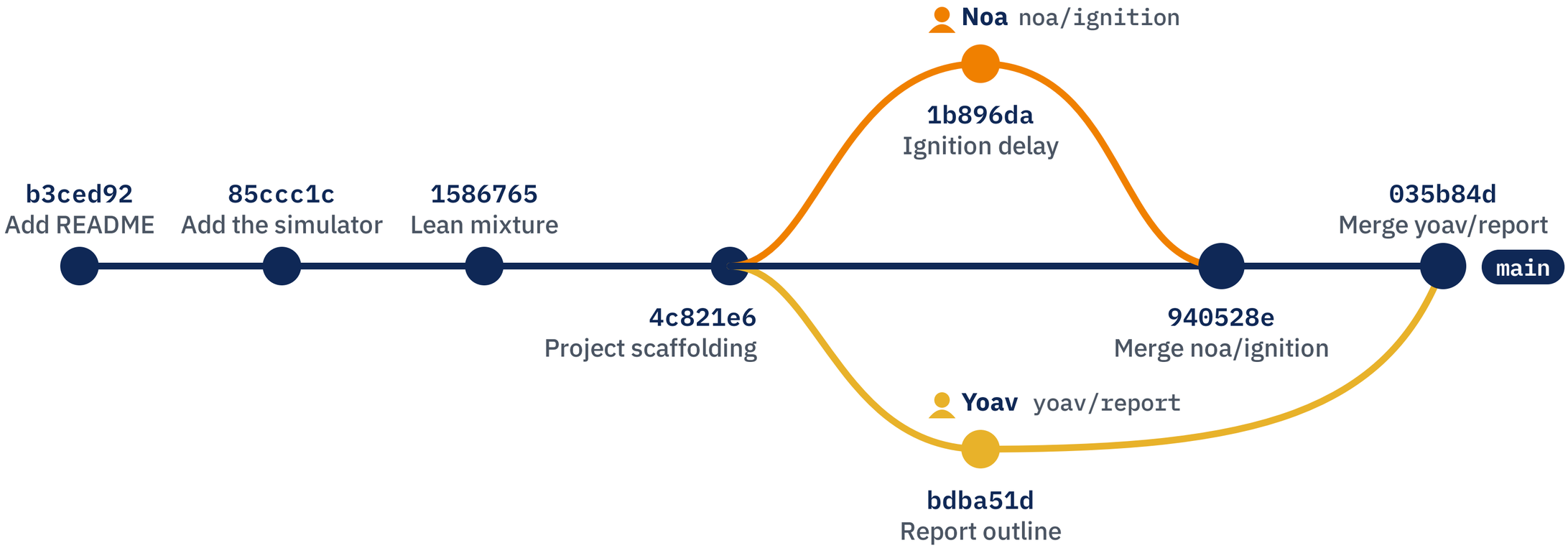
history (git log)



Same folder. Different branch, different files.

git: working as a team

two people, two branches, one main



Everyone works on a branch. main only moves by merging.

On GitHub, a merge into main is a **pull request**. Merge **conflicts**: Week 2.

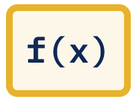
Outline

01	Syllabus	the course, the weeks, the milestones
02	Concept	the 10× claim, three approaches, your environment
03	git	organizes and protects the code
04	**Live demo**	a real example ←

The final project

You will build an **Approach C** system.

The contract, in one line: a ground, a loop, a boundary.



A ground

Deterministic and tested. No model in here.



A loop

An agent reasons, runs, reads, adjusts.



A boundary

The only way between model and ground.

This week's increment: the problem and the candidate ground.
The last block of today sets it out, and lists everything you hand in.

Long form: FINAL-PROJECT.md §1 to §3

This week, your project

Increment M1, due in Week 2

`proposal-draft.md`

The problem

The candidate ground

Your first guess at where the line is

The guess is meant to be wrong.
On paper now, corrected in W6, not discovered in W11.

REFERENCE

Qualifying a ground

Six criteria, all required

01	**Importable, with a callable API**
02	**Free or open source**
03	**Deterministic**
04	**Bounded and fast**
05	**Testable**
06	**Hard enough to need reasoning, not so hard the loop stalls**

Criterion 6 is the one that fails people, and it fails late.

One call gives the final answer? That is a script, not a project.

Nothing the loop does makes progress? It demonstrates nothing.

Long form: PROJECT-QUALIFICATION.md · bring your idea to the W6 exercise

Before the next exercise

The Pre-work page, finished

Python you can read	A real Unix shell	uv installed	GitHub, with 2FA

A prerequisite, not a module.

The W1 and W2 exercises **verify** it, with TAs in the room.
They do not teach it from zero. The Pre-work page does.

W2 ends in the Setup Gate:
`check-sandbox.sh` printing `GATE: PASS`.

What you hand in, all semester

`src/`	ground, boundary, loop
`tests/`	green in CI
`evals/`	does the loop converge
`AGENTS.md`	GLOSSARY.md, one skill
`runs/`	including the failures

`docs/report.md`	
`demo video`	5 minutes
`peer review`	one
`defence pack`	one page

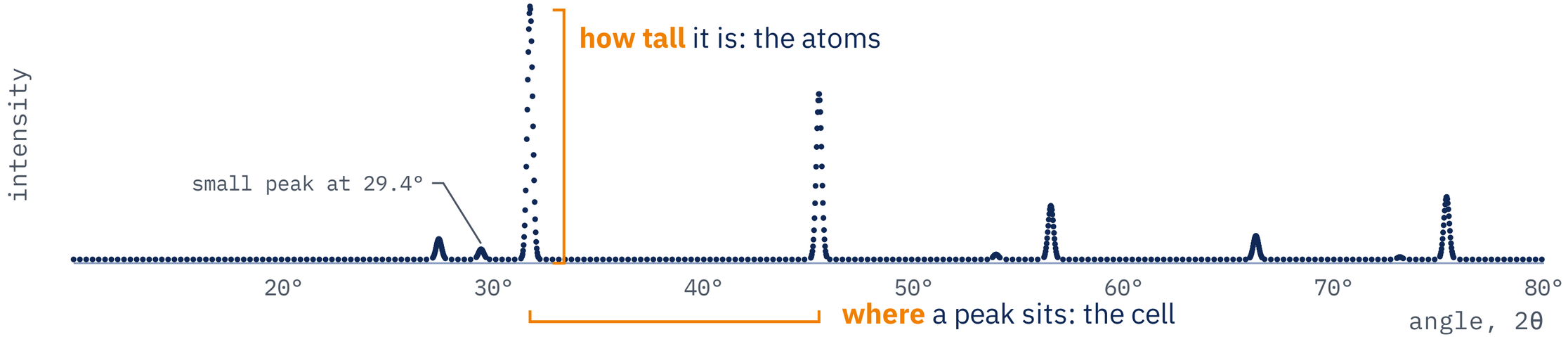
In W13 a randomly selected few **defend it live,** before the whole course staff. **Prepare as though it is you.**

Examined on: can you **explain, defend and verify** what you shipped?
We assume you used agents heavily. That is the subject.



Example: a crystal from its powder pattern

mock data, real physics



The experiment

X-rays hit a powder of tiny crystals. Each set of atomic planes reflects at its own angle. The pattern is a fingerprint, not a picture.

Two halves

Cell first, from the peak *positions*. Atoms second, from the peak *heights*.

The loop

The model proposes a crystal. The ground computes its pattern. The model reads the difference and proposes again.

A simulated rock-salt pattern (pymatgen, Cu Kα) plus one small impurity peak. Not a measured sample.

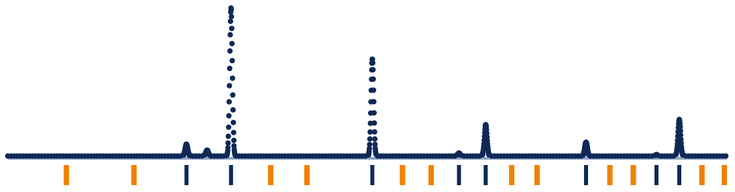
The loop, four turns

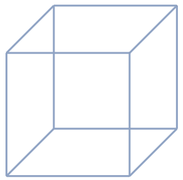
propose, compute, read the misfit, propose again

measured · computed by the ground · difference · allowed position · heavy atom · light atom

1 · Index the peaks ALGORITHM

positions only



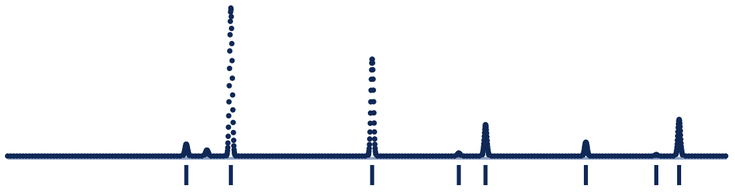


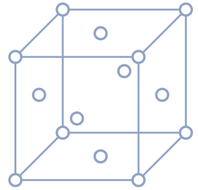
cubic, a = 5.64 Å

Every main peak fits a cubic cell, a = 5.64 Å. A primitive cell also allows peaks where nothing was measured (**orange**): the cell is centred.

2 · Choose the centring ALGORITHM

positions only



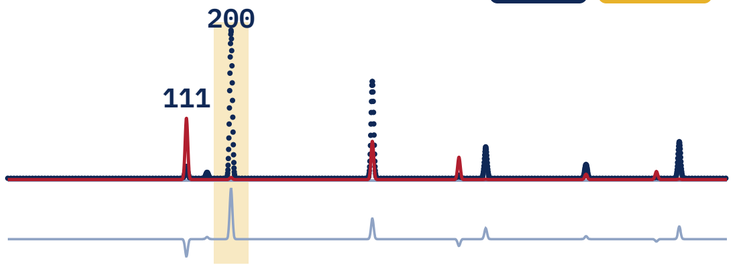


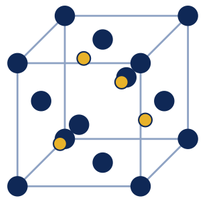
F lattice, no atoms

F-centring (hkl all odd or all even) removes exactly those. The lattice is settled. No atoms yet, so **positions only, no heights**.

3 · Place the atoms: first try MODEL GROUND

misfit 91%



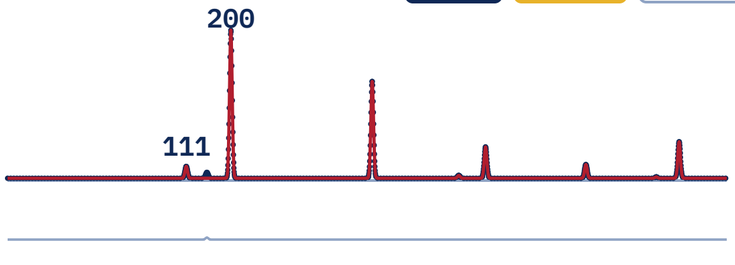


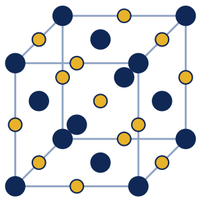
tetrahedral holes

Zinc blende: the light atom in the tetrahedral holes. **200 far too weak, 111 far too strong.**

4 · Move the light atom MODEL GROUND ALGORITHM

misfit 3%





octahedral holes

The radius ratio wants six neighbours: rock salt. The difference goes flat. The 29.4° peak is **an impurity**, not a reason to change the structure.

Where the line fell

the crystal example, step by step

STEP	WHO	WHY
Find the cell	ALGORITHM	Solved decades ago (DICVOL, TREOR)
Compute any pattern	GROUND	Physics: pymatgen. Every number on screen came from here
Choose the structure	MODEL	A chemical hypothesis, not a point on a grid
Fit the positions	ALGORITHM	Least squares beats any model
Call 29.4° an impurity	MODEL	Judgement about what *not* to fit. A for-loop overfits it

The model earns its place in two rows:
choosing the structure, and declining to fit the impurity.

MODEL reasons GROUND computes, deterministic ALGORITHM fits or searches, no judgement

NEXT LECTURE

Week 2: LLM science, and what each mechanic implies about failure

Transformer, next-token prediction, the context window, scaling.
For each mechanic: the specific way it fails.

The exercise before it is Environment and git.
Bring the machine you will work on.